

LAPORAN PRAKTIKUM

STRUKTUR DATA

DOUBLE LINKED LIST

PEKAN 6



Disusun Oleh :

KINAYA NOVRYA MANDA

(2511531016)

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis sampaikan kehadirat Allah SWT. Salawat dan salam disampaikan kepada Nabi Muhammad SAW. Karena thaufik dan hidayah-Nya, laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban dari kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar serta penerapan bahasa pemrograman Java.

Melalui praktikum ini, penulis memperoleh pengalaman langsung dalam memahami Implementasi Double Linked List menggunakan Java. Diharapkan laporan ini dapat memberikan gambaran yang jelas mengenai materi yang telah dipelajari serta hasil dari percobaan yang dilakukan selama praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi isi maupun penyajiannya. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu, asisten laboratorium, serta semua pihak yang telah memberikan bimbingan, arahan, dan dukungan sehingga laporan praktikum ini dapat tersusun. Semoga laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami dasar-dasar pemrograman Java.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
DAFTAR LAMPIRAN.....	iii
BAB 1 PENDAHULUAN	
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	2
1.3 Manfaat Praktikum.....	2
BAB 2 PEMBAHASAN	
2.1 Dasar Teori.....	3
2.2 Kode Program.....	6
2.3 Langkah Kerja.....	11
2.4 Analisis Hasil.....	24
BAB 3 KESIMPULAN	
3.1 Hasil Praktikum.....	25
3.2 Saran Pengembangan.....	28
DAFTAR KEPUSTAKAAN.....	29
TUGAS PRAKTIKUM PEKAN 6.....	30

DAFTAR LAMPIRAN

Kode Program 2.1 (Kode Program NodeDLL).....	6
Kode Program 2.2 (Kode Program InsertDLL).....	7
Kode Program 2.3 (Kode Program HapusDLL).....	9
Kode Program 2.4 (Kode Program PenelusuranDLL).....	10
Gambar 2.1 (Langkah 1 NodeDLL).....	11
Gambar 2.2 (Langkah 2 NodeDLL).....	11
Gambar 2.3 (Langkah 3 NodeDLL).....	12
Gambar 2.4 (Langkah 1 InsertDLL).....	12
Gambar 2.5 (Langkah 2 InsertDLL).....	13
Gambar 2.6 (Langkah 3 InsertDLL).....	13
Gambar 2.7 (Langkah 4 InsertDLL).....	13
Gambar 2.8 (Langkah 5 InsertDLL).....	14
Gambar 2.9 (Langkah 6 InsertDLL).....	14
Gambar 2.10 (Langkah 7 InsertDLL).....	15
Gambar 2.11 (Langkah 8 InsertDLL).....	15
Gambar 2.12 (Langkah 9 InsertDLL).....	15
Gambar 2.13 (Langkah 10 InsertDLL).....	16
Gambar 2.14 (Langkah 11 InsertDLL).....	16
Gambar 2.15 (Langkah 1 HapusDLL).....	16
Gambar 2.16 (Langkah 2 HapusDLL).....	17
Gambar 2.17 (Langkah 3 HapusDLL).....	17
Gambar 2.18 (Langkah 4 HapusDLL).....	17
Gambar 2.19 (Langkah 5 HapusDLL).....	18

Gambar 2.20 (Langkah 6 HapusDLL).....	18
Gambar 2.21 (Langkah 7 HapusDLL).....	18
Gambar 2.22 (Langkah 8 HapusDLL).....	19
Gambar 2.23 (Langkah 9 HapusDLL).....	19
Gambar 2.24 (Langkah 10 HapusDLL).....	19
Gambar 2.25 (Langkah 11 HapusDLL).....	20
Gambar 2.26 (Langkah 12 HapusDLL).....	20
Gambar 2.27 (Langkah 1 PenelusuranDLL).....	21
Gambar 2.28 (Langkah 2 PenelusuranDLL).....	21
Gambar 2.29 (Langkah 3 PenelusuranDLL).....	22
Gambar 2.30 (Langkah 4 PenelusuranDLL).....	22
Gambar 2.31 (Langkah 5 PenelusuranDLL).....	22
Gambar 2.32 (Langkah 6 PenelusuranDLL).....	23
Gambar 2.33 (Langkah 7 PenelusuranDLL).....	23
Gambar 2.34 (Langkah 8 PenelusuranDLL).....	23
Gambar 3.1 (Hasil Praktikum InsertDLL).....	25
Gambar 3.2 (Hasil Praktikum HapusDLL).....	26
Gambar 3.3 (Hasil Praktikum PenelusuranDLL).....	27

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pemrograman karena digunakan untuk mengatur dan mengelola data agar dapat diproses secara lebih efisien. Salah satu struktur data yang sering digunakan adalah linked list, yaitu struktur data yang menyimpan elemen dalam bentuk node yang saling terhubung. Berbeda dengan array yang memiliki ukuran tetap, linked list bersifat dinamis sehingga penambahan maupun penghapusan data dapat dilakukan tanpa harus memindahkan seluruh elemen. Dalam perkembangannya, linked list dibedakan menjadi beberapa jenis, salah satunya adalah Double Linked List (DLL).

Double Linked List merupakan linked list yang setiap node-nya memiliki dua pointer, yaitu pointer menuju node berikutnya (next) dan pointer menuju node sebelumnya (prev). Adanya dua arah hubungan ini membuat proses penelusuran data menjadi lebih fleksibel karena data dapat diakses dari depan ke belakang maupun dari belakang ke depan. Selain itu, proses penyisipan dan penghapusan node juga menjadi lebih mudah karena program dapat langsung menghubungkan node sebelumnya dan node sesudahnya tanpa harus menelusuri ulang dari awal. Oleh karena itu, DLL sering digunakan pada berbagai sistem seperti navigasi browser, playlist musik, hingga fitur undo dan redo pada aplikasi.

Melalui praktikum ini, mahasiswa mempelajari cara kerja DLL mulai dari pembuatan node, proses penambahan node, penghapusan node, hingga penelusuran data secara maju dan mundur menggunakan bahasa pemrograman Java. Praktikum ini tidak hanya membantu memahami konsep dasar struktur data, tetapi juga melatih kemampuan dalam mengelola pointer antar node agar susunan data tetap terhubung dengan benar. Dengan memahami implementasi DLL secara langsung, mahasiswa diharapkan dapat lebih memahami penerapan struktur data dinamis dalam pemrograman.

1.2 Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami konsep dasar DLL beserta penggunaan pointer next dan prev pada setiap node.
2. Mengimplementasikan operasi dasar DLL seperti penambahan, penghapusan, dan penelusuran node menggunakan bahasa pemrograman Java.
3. Melatih kemampuan mahasiswa dalam mengelola hubungan antar node agar struktur data tetap tersusun dengan benar setelah dilakukan manipulasi data.

1.3 Manfaat Praktikum

Manfaat dilakukannya praktikum ini adalah sebagai berikut :

1. Mahasiswa dapat memahami cara kerja struktur data dinamis yang memungkinkan data ditambah dan dihapus secara fleksibel.
2. Mahasiswa memperoleh pengalaman dalam mengimplementasikan DLL secara langsung menggunakan Java.
3. Mahasiswa dapat meningkatkan pemahaman mengenai penggunaan pointer dan hubungan antar node dalam pengolahan data pada struktur linked list.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Double Linked List (DLL) merupakan salah satu jenis struktur data linked list yang setiap node-nya memiliki dua buah pointer, yaitu pointer ke node berikutnya (next) dan pointer ke node sebelumnya (prev). Struktur data ini digunakan untuk menyimpan sekumpulan data secara dinamis, sehingga ukuran data dapat bertambah maupun berkurang selama program berjalan. Berbeda dengan array yang memiliki kapasitas tetap, DLL lebih fleksibel karena penambahan dan penghapusan data tidak memerlukan pergeseran elemen lain di memori. Oleh karena itu, DLL sering digunakan pada program yang membutuhkan navigasi data dua arah secara efisien.

Pada Double Linked List, setiap node terdiri dari tiga bagian utama, yaitu data, pointer next, dan pointer prev. Pointer next digunakan untuk menunjuk node setelahnya, sedangkan pointer prev digunakan untuk menunjuk node sebelumnya. Node pertama disebut head, sedangkan node terakhir disebut tail. Adanya dua arah hubungan antar node membuat proses traversal atau penelusuran data dapat dilakukan dari depan ke belakang maupun dari belakang ke depan. Konsep ini membuat DLL lebih unggul dibanding SLL dalam beberapa kondisi, terutama saat proses penghapusan atau penelusuran mundur dilakukan.

Secara umum, DLL memiliki beberapa operasi dasar, yaitu:

1. insert di awal (insert beginning), untuk menambahkan node pada bagian depan linked list,
2. insert di akhir (insert end), untuk menambahkan node pada bagian belakang linked list,
3. insert pada posisi tertentu, untuk menyisipkan node di tengah linked list,
4. delete head, untuk menghapus node pertama,
5. delete tail, untuk menghapus node terakhir,

6. delete posisi tertentu, untuk menghapus node pada lokasi tertentu,
7. traversal maju, untuk menelusuri node dari head menuju tail,
8. traversal mundur, untuk menelusuri node dari tail menuju head.

Pada proses insert di awal, langkah yang dilakukan adalah:

1. membuat node baru,
2. menghubungkan pointer next node baru ke head lama,
3. mengubah pointer prev head lama ke node baru,
4. memindahkan head ke node baru.

Sedangkan pada proses insert di akhir, langkah-langkahnya yaitu:

1. membuat node baru,
2. menelusuri node hingga node terakhir,
3. menghubungkan pointer next node terakhir ke node baru,
4. mengatur pointer prev node baru ke node terakhir.

Untuk proses delete head, langkah yang dilakukan yaitu:

1. mengecek apakah linked list kosong,
2. memindahkan head ke node berikutnya,
3. mengubah pointer prev head baru menjadi null.

Sementara itu, pada delete tail langkahnya adalah:

1. menelusuri hingga node terakhir,
2. memutus hubungan node terakhir dengan node sebelumnya,
3. mengubah pointer next node sebelumnya menjadi null.

Pada traversal maju, penelusuran dilakukan menggunakan pointer next mulai dari head hingga mencapai null. Sedangkan traversal mundur dilakukan menggunakan pointer prev mulai dari tail menuju head. Dengan adanya dua arah penelusuran ini, Double Linked List menjadi lebih fleksibel dalam pengolahan data dibandingkan Single Linked List.

Dalam implementasinya, DLL banyak digunakan pada berbagai aplikasi seperti fitur undo-redo pada editor, navigasi halaman browser, playlist musik, hingga sistem navigasi data yang memerlukan perpindahan maju dan mundur secara cepat. Dengan demikian, DLL menjadi salah satu struktur data penting yang membantu proses pengelolaan data secara dinamis dan efisien dalam pemrograman.

2.2 Kode Program

2.2.1 NodeDLL_2511531016

```
package pekan6_2511531016;

public class NodeDLL_2511531016 {
    // Mendefinisikan kelas node
    int data_1016; // data
    NodeDLL_2511531016 next_1016; // pointer ke next node
    NodeDLL_2511531016 prev_1016; // pointer ke previous node
    // Konstruktor
    public NodeDLL_2511531016(int data_1016) {
        this.data_1016 = data_1016;
        this.next_1016 = null;
        this.prev_1016 = null;
    }
}
```

Kode Program 2.1 (Kode Program NodeDLL)

2.2.2 InsertDLL_2511531016

```
package pekan6_2511531016;

public class InsertDLL_2511531016 {
    // Menambahkan node di awal DLL
    static NodeDLL_2511531016 insertBegin_1016 (NodeDLL_2511531016 head_1016, int data_1016) {
        // Buat node baru
        NodeDLL_2511531016 new_node = new NodeDLL_2511531016 (data_1016);
        // Jadikan pointer nextnya head
        new_node.next_1016 = head_1016;
        // Jadikan pointer prev head ke new_node
        if (head_1016 != null) {
            head_1016.prev_1016 = new_node;
        }
        return new_node;
    }

    // Fungsi menambahkan node di akhir
    public static NodeDLL_2511531016 insertEnd_1016 (NodeDLL_2511531016 head_1016, int newData_1016) {
        // Buat node baru
        NodeDLL_2511531016 newNode = new NodeDLL_2511531016 (newData_1016);
        // Jika d11 null jadikan head
        if (head_1016 == null) {
            head_1016 = newNode;
        }
        else {
            NodeDLL_2511531016 curr_1016 = head_1016;
            while (curr_1016.next_1016 != null) {
                curr_1016 = curr_1016.next_1016;
            }
            curr_1016.next_1016 = newNode;
            newNode.prev_1016 = curr_1016;
        }
        return head_1016;
    }
}
```

```

// Fungsi menambahkan node di posisi tertentu
public static NodeDLL_2511531016 insertAtPosition_1016 (NodeDLL_2511531016 head_1016, int pos_1016, int new_data) {
    // Buat node baru
    NodeDLL_2511531016 new_node = new NodeDLL_2511531016 (new_data);
    if (pos_1016 == 1) {
        new_node.next_1016 = head_1016;
        if (head_1016 != null) {
            head_1016.prev_1016 = new_node;
        }
        head_1016 = new_node;
        return head_1016;
    }
    NodeDLL_2511531016 curr_1016 = head_1016;
    for (int i_1016 = 1; i_1016 < pos_1016 - 1 && curr_1016 != null; ++i_1016) {
        curr_1016 = curr_1016.next_1016;
    }
    if (curr_1016 == null) {
        System.out.println ("Posisi tidak ada");
        return head_1016;
    }
    new_node.prev_1016 = curr_1016;
    new_node.next_1016 = curr_1016.next_1016;
    curr_1016.next_1016 = new_node;
    if (new_node.next_1016 != null) {
        new_node.next_1016.prev_1016 = new_node;
    }
    return head_1016;
}

public static void printList_1016 (NodeDLL_2511531016 head_1016) {
    NodeDLL_2511531016 curr_1016 = head_1016;
    while (curr_1016 != null) {
        System.out.print(curr_1016.data_1016 + " <-> ");
        curr_1016 = curr_1016.next_1016;
    }
    System.out.println();
}

public static void main(String[] args) {
    // Membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511531016 head_1016 = new NodeDLL_2511531016(2);
    head_1016.next_1016 = new NodeDLL_2511531016(3);
    head_1016.next_1016.prev_1016 = head_1016;
    head_1016.next_1016.next_1016 = new NodeDLL_2511531016(5);
    head_1016.next_1016.next_1016.prev_1016 = head_1016.next_1016;
    // Cetak DLL awal
    System.out.print("DLL Awal: ");
    printList_1016(head_1016);
    // Tambah 1 di awal
    head_1016 = insertBegin_1016(head_1016, 1);
    System.out.print("Simpul 1 ditambah di awal: ");
    printList_1016(head_1016);
    // Tambah 6 di akhir
    System.out.print("Simpul 6 ditambah di akhir: ");
    int data_1016 = 6;
    head_1016 = insertEnd_1016(head_1016, data_1016);
    printList_1016(head_1016);
    // Menambah node 4 di posisi 4
    System.out.print("Tambah node 4 di posisi 4: ");
    int data2 = 4;
    int pos_1016 = 4;
    head_1016 = insertAtPosition_1016(head_1016, pos_1016, data2);
    printList_1016(head_1016);
}
}

```

Kode Program 2.2 (Kode Program InsertDLL)

2.2.3 HapusDLL_2511531016

```
package pekan6_2511531016;

public class HapusDLL_2511531016 {
    // Fungsi menghapus node awal
    public static NodeDLL_2511531016 delHead_1016 (NodeDLL_2511531016 head_1016) {
        if (head_1016 == null) {
            return null;
        }
        NodeDLL_2511531016 temp_1016 = head_1016;
        head_1016 = head_1016.next_1016;
        if (head_1016 != null) {
            head_1016.prev_1016 = null;
        }
        return head_1016;
    }

    // Fungsi menghapus di akhir
    public static NodeDLL_2511531016 dellast_1016 (NodeDLL_2511531016 head_1016) {
        if (head_1016 == null) {
            return null;
        }
        if (head_1016.next_1016 == null) {
            return null;
        }
        NodeDLL_2511531016 curr_1016 = head_1016;
        while (curr_1016.next_1016 != null) {
            curr_1016 = curr_1016.next_1016;
        }

        // Update pointer previous node
        if (curr_1016.prev_1016 != null) {
            curr_1016.prev_1016.next_1016 = null;
        }
        return head_1016;
    }

    // Fungsi menghapus node posisi tertentu
    public static NodeDLL_2511531016 delPos_1016 (NodeDLL_2511531016 head_1016, int pos_1016) {
        // Jika DLL kosong
        if (head_1016 == null) {
            return head_1016;
        }
        NodeDLL_2511531016 curr_1016 = head_1016;
        // Telusuri sampai ke node yang akan dihapus
        for (int i_1016 = 1; curr_1016 != null && i_1016 < pos_1016; ++i_1016) {
            curr_1016 = curr_1016.next_1016;
        }
        // Jika posisi tidak ditemukan
        if (curr_1016 == null) {
            return head_1016;
        }
        // Update pointer
        if (curr_1016.prev_1016 != null) {
            curr_1016.prev_1016.next_1016 = curr_1016.next_1016;
        }
        if (curr_1016.next_1016 != null) {
            curr_1016.next_1016.prev_1016 = curr_1016.prev_1016;
        }
        // Jika yang dihapus head
        if (head_1016 == curr_1016) {
            head_1016 = curr_1016.next_1016;
        }
        return head_1016;
    }

    // Fungsi mencetak DLL
    public static void printList_1016 (NodeDLL_2511531016 head_1016) {
        NodeDLL_2511531016 curr_1016 = head_1016;
        while (curr_1016 != null) {
            System.out.print(curr_1016.data_1016 + " ");
            curr_1016 = curr_1016.next_1016;
        }
    }
}
```

```

        System.out.println();
    }

    public static void main(String[] args) {
        // Buat sebuah DLL
        NodeDLL_2511531016 head_1016 = new NodeDLL_2511531016(1);
        head_1016.next_1016 = new NodeDLL_2511531016(2);
        head_1016.next_1016.next_1016 = new NodeDLL_2511531016(3);
        head_1016.next_1016.next_1016.prev_1016 = head_1016.next_1016;
        head_1016.next_1016.next_1016.next_1016 = new NodeDLL_2511531016(4);
        head_1016.next_1016.next_1016.next_1016.prev_1016 = head_1016.next_1016.next_1016;
        head_1016.next_1016.next_1016.next_1016.next_1016 = new NodeDLL_2511531016(5);
        head_1016.next_1016.next_1016.next_1016.next_1016.prev_1016 = head_1016.next_1016.next_1016.next_1016;

        System.out.print("DLL Awal: ");
        printList_1016(head_1016);

        System.out.print("Setelah head dihapus: ");
        head_1016 = delHead_1016(head_1016);
        printList_1016(head_1016);

        System.out.print("Setelah node terakhir dihapus: ");
        head_1016 = dellLast_1016(head_1016);
        printList_1016(head_1016);

        System.out.print("menghapus node ke 2: ");
        head_1016 = delPos_1016(head_1016, 2);

        printList_1016(head_1016);
    }

```

Kode Program 2.3 (Kode Program HapusDLL)

2.2.4 PenelusuranDLL_2511531016

`package` pekan6_2511531016;

```

public class PenelusuranDLL_2511531016 {
    // Fungsi penelusuran maju
    static void forwardTraversal_1016 (NodeDLL_2511531016 head_1016) {
        // Memulai penelusuran dari head
        NodeDLL_2511531016 curr_1016 = head_1016;
        // Lanjutkan sampai akhir
        while (curr_1016 != null) {
            // Print data
            System.out.print(curr_1016.data_1016 + " <-> ");
            // Pindah ke node berikutnya
            curr_1016 = curr_1016.next_1016;
        }
        // Print spasi
        System.out.println();
    }

    // Fungsi penelusuran mundur
    static void backwardTraversal_1016 (NodeDLL_2511531016 tail_1016) {
        // Mulai dari akhir
        NodeDLL_2511531016 curr_1016 = tail_1016;
        // Lanjut sampai head
        while (curr_1016 != null) {
            // Cetak data
            System.out.print(curr_1016.data_1016 + " <-> ");
            // Pindah ke node sebelumnya
            curr_1016 = curr_1016.prev_1016;
        }
        // Cetak spasi
        System.out.println();
    }

    public static void main(String[] args) {
        // Cetak DLL
    }
}

```

```

NodeDLL_2511531016 head_1016 = new NodeDLL_2511531016(1);
NodeDLL_2511531016 second_1016 = new NodeDLL_2511531016(2);
NodeDLL_2511531016 third_1016 = new NodeDLL_2511531016(3);

head_1016.next_1016 = second_1016;
second_1016.prev_1016 = head_1016;
second_1016.next_1016 = third_1016;
third_1016.prev_1016 = second_1016;

System.out.println("Penelusuran maju:");
forwardTraversal_1016(head_1016);

System.out.println("Penelusuran mundur:");
backwardTraversal_1016(third_1016);
}
}

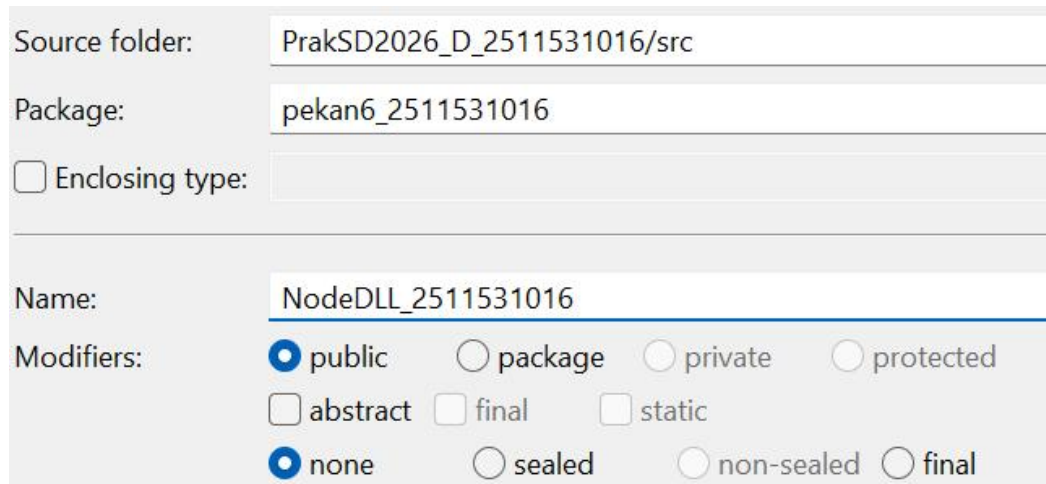
```

Kode Program 2.4 (Kode Program PenelusuranDLL)

2.3 Langkah Kerja

2.3.1 NodeDLL_2511531016

1. Membuat kelas baru bernama *NodeDLL_2511531016* dalam *package pekan6_2511531016*.



Source folder: PrakSD2026_D_2511531016/src

Package: pekan6_2511531016

☐ Enclosing type:

Name: NodeDLL_2511531016

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.1 (Langkah 1 NodeDLL)

2. Program diawali dengan deklarasi package dan pembuatan class. Lalu, program mendefinisikan kelas *NodeDLL_2511531016* sebagai node pada Double Linked List, di mana *data_1016* digunakan untuk menyimpan data, sedangkan *next_1016* dan *prev_1016* berfungsi sebagai penunjuk ke node berikutnya dan sebelumnya.

```
package pekan6_2511531016;  
  
public class NodeDLL_2511531016 {  
    // Mendefinisikan kelas node  
    int data_1016; // data  
    NodeDLL_2511531016 next_1016; // pointer ke next node  
    NodeDLL_2511531016 prev_1016; // pointer ke previous node  
}
```

Gambar 2.2 (Langkah 2 NodeDLL)

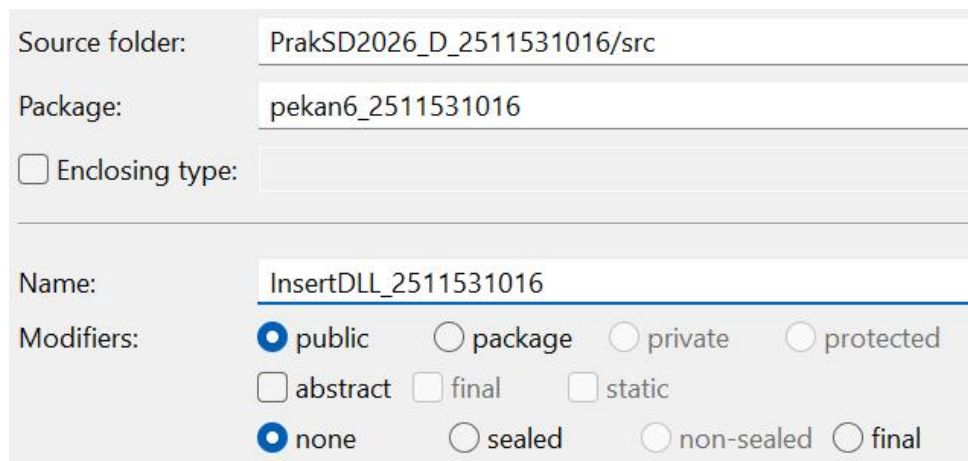
3. Kemudian, buat konstruktor untuk menginisialisasi nilai data pada node sekaligus mengatur pointer next_1016 dan prev_1016 menjadi null karena node belum terhubung ke node lain.

```
// Konstruktor
public NodeDLL_2511531016(int data_1016) {
    this.data_1016 = data_1016;
    this.next_1016 = null;
    this.prev_1016 = null;
}
```

Gambar 2.3 (Langkah 3 NodeDLL)

2.3.2 InsertDLL_2511531016

1. Membuat kelas baru bernama *InsertDLL_2511531016* dalam package *pekan6_2511531016*.



Source folder:	PrakSD2026_D_2511531016/src
Package:	pekan6_2511531016
☐ Enclosing type:	
Name:	InsertDLL_2511531016
Modifiers:	☒ public ☐ package ☐ private ☐ protected ☐ abstract ☐ final ☐ static ☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.4 (Langkah 1 InsertDLL)

2. Deklarasikan package dan class. Lalu, buat method insertBegin_1016 untuk menambahkan node di awal Double Linked List dengan membuat node baru, lalu menghubungkan pointer next_1016 ke head lama dan pointer prev_1016 head lama ke node baru.

```
package pekan6_2511531016;

public class InsertDLL_2511531016 {
```

```
// Menambahkan node di awal DLL
static NodeDLL_2511531016 insertBegin_1016 (NodeDLL_2511531016
head_1016, int data_1016) {
// Buat node baru
NodeDLL_2511531016 new_node = new NodeDLL_2511531016 (data_1016);
// Jadikan pointer nextnya head
new_node.next_1016 = head_1016;
// Jadikan pointer prev head ke new_node
if (head_1016 != null) {
head_1016.prev_1016 = new_node;
}
return new_node;
}
```

Gambar 2.5 (Langkah 2 InsertDLL)

3. Selanjutnya, buat method insertEnd_1016 untuk menambahkan node di akhir list dengan membuat node baru, lalu cek apakah Double Linked List masih kosong sehingga node baru bisa langsung dijadikan head.

```
// Fungsi menambahkan node di akhir
public static NodeDLL_2511531016 insertEnd_1016 (NodeDLL_2511531016
head_1016, int newData_1016) {
// Buat node baru
NodeDLL_2511531016 newNode = new NodeDLL_2511531016 (newData_1016);
// Jika d11 null jadikan head
if (head_1016 == null) {
head_1016 = newNode;
}
```

Gambar 2.6 (Langkah 3 InsertDLL)

4. Jika list tidak kosong maka program menelusuri hingga node terakhir, kemudian hubungkan node terakhir dengan node baru menggunakan pointer next_1016 dan prev_1016.

```
else {
NodeDLL_2511531016 curr_1016 = head_1016;
while (curr_1016.next_1016 != null) {
curr_1016 = curr_1016.next_1016;
}
curr_1016.next_1016 = newNode;
newNode.prev_1016 = curr_1016;
}
return head_1016;
}
```

Gambar 2.7 (Langkah 4 InsertDLL)

5. Kemudian, buat method `insertAtPosition_1016` untuk menambahkan node pada posisi tertentu, termasuk menangani kondisi jika posisi yang dimasukkan adalah posisi pertama.

```
// Fungsi menambahkan node di posisi tertentu
public static NodeDLL_2511531016 insertAtPosition_1016
(NodeDLL_2511531016 head_1016, int pos_1016, int new_data) {
// Buat node baru
NodeDLL_2511531016 new_node = new NodeDLL_2511531016 (new_data);
if (pos_1016 == 1) {
new_node.next_1016 = head_1016;
if (head_1016 != null) {
head_1016.prev_1016 = new_node;
}
head_1016 = new_node;
return head_1016;
}
NodeDLL_2511531016 curr_1016 = head_1016;
```

Gambar 2.8 (Langkah 5 InsertDLL)

6. Telusuri node hingga posisi yang diinginkan menggunakan perulangan, lalu periksa apakah posisi tersebut tersedia atau tidak.

```
for (int i_1016 = 1; i_1016 < pos_1016 - 1 && curr_1016 != null;
++i_1016) {
curr_1016 = curr_1016.next_1016;
}
if (curr_1016 == null) {
System.out.println ("Posisi tidak ada");
return head_1016;
}
```

Gambar 2.9 (Langkah 6 InsertDLL)

7. Setelah posisi ditemukan, program menyisipkan node baru dengan mengatur hubungan pointer `prev_1016` dan `next_1016` agar struktur Double Linked List tetap terhubung dengan benar.

```
new_node.prev_1016 = curr_1016;
new_node.next_1016 = curr_1016.next_1016;
curr_1016.next_1016 = new_node;
if (new_node.next_1016 != null) {
```

```

new_node.next_1016.prev_1016 = new_node;
}
return head_1016;

```

Gambar 2.10 (Langkah 7 InsertDLL)

8. Buat method `printList_1016` untuk menampilkan seluruh isi Double Linked List dengan menelusuri node dari awal hingga akhir secara berurutan.

```

public static void printList_1016 (NodeDLL_2511531016 head_1016) {
NodeDLL_2511531016 curr_1016 = head_1016;
while (curr_1016 != null) {
System.out.print(curr_1016.data_1016 + " <-> ");
curr_1016 = curr_1016.next_1016;
}
System.out.println();
}

```

Gambar 2.11 (Langkah 8 InsertDLL)

9. Selanjutnya, pada method `main`, buat program Double Linked List awal berisi node 2, 3, dan 5, lalu hubungkan masing-masing node menggunakan pointer `next_1016` dan `prev_1016`, kemudian cetak list awal.

```

public static void main(String[] args) {
// Membuat d11 2 <-> 3 <-> 5
NodeDLL_2511531016 head_1016 = new NodeDLL_2511531016(2);
head_1016.next_1016 = new NodeDLL_2511531016(3);
head_1016.next_1016.prev_1016 = head_1016;
head_1016.next_1016.next_1016 = new NodeDLL_2511531016(5);
head_1016.next_1016.next_1016.prev_1016 = head_1016.next_1016;
// Cetak DLL awal
System.out.print("DLL Awal: ");
printList_1016(head_1016);
}

```

Gambar 2.12 (Langkah 9 InsertDLL)

10. Tambahkan node bernilai 1 di awal dan node bernilai 6 di akhir menggunakan method yang telah dibuat, lalu cetak hasil setelah setiap penambahan.

```
// Tambah 1 di awal
head_1016 = insertBegin_1016(head_1016, 1);
System.out.print("Simpul 1 ditambah di awal: ");
printList_1016(head_1016);
// Tambah 6 di akhir
System.out.print("Simpul 6 ditambah di akhir: ");
int data_1016 = 6;
head_1016 = insertEnd_1016(head_1016, data_1016);
printList_1016(head_1016);
```

Gambar 2.13 (Langkah 10 InsertDLL)

11. Terakhir, tambahkan node bernilai 4 pada posisi ke-4 menggunakan method `insertAtPosition_1016`, lalu tampilkan hasil akhir Double Linked List setelah semua proses penambahan selesai.

```
// Menambah node 4 di posisi 4
System.out.print("Tambah node 4 di posisi 4: ");
int data2 = 4;
int pos_1016 = 4;
head_1016 = insertAtPosition_1016(head_1016, pos_1016, data2);
printList_1016(head_1016);
```

Gambar 2.14 (Langkah 11 InsertDLL)

2.3.3 HapusDLL_2511531016

1. Membuat kelas baru bernama *HapusDLL_2511531016* dalam package *pekan6_2511531016*.

Source folder:	PrakSD2026_D_2511531016/src
Package:	pekan6_2511531016
☐ Enclosing type:	
Name:	HapusDLL_2511531016
Modifiers:	☒ public ☐ package ☐ private ☐ protected ☐ abstract ☐ final ☐ static ☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.15 (Langkah 1 HapusDLL)

2. Program dimulai dengan deklarasi package dan class. Lalu, buat method `delHead_1016()` untuk menghapus node pertama. Jika linked list kosong maka method akan mengembalikan null sehingga program tidak mengalami error saat proses penghapusan dilakukan.

```
package pekan6_2511531016;

public class HapusDLL_2511531016 {
    // Fungsi menghapus node awal
    public static NodeDLL_2511531016 delHead_1016 (NodeDLL_2511531016
head_1016) {
    if (head_1016 == null) {
        return null;
    }
}
```

Gambar 2.16 (Langkah 2 HapusDLL)

3. Selanjutnya, pindahkan head ke node berikutnya dan ubah pointer `prev_1016` menjadi null agar node baru menjadi head pertama pada DLL.

```
NodeDLL_2511531016 temp_1016 = head_1016;
head_1016 = head_1016.next_1016;
if (head_1016 != null) {
    head_1016.prev_1016 = null;
}
return head_1016;
```

Gambar 2.17 (Langkah 3 HapusDLL)

4. Buat method `delLast_1016()` untuk menghapus node terakhir pada DLL dengan cara menelusuri node dari awal hingga mencapai node paling akhir.

```
// Fungsi menghapus di akhir
public static NodeDLL_2511531016 delLast_1016 (NodeDLL_2511531016
head_1016) {
    if (head_1016 == null) {
        return null;
    }
    if (head_1016.next_1016 == null) {
        return null;
    }
    NodeDLL_2511531016 curr_1016 = head_1016;
    while (curr_1016.next_1016 != null) {
        curr_1016 = curr_1016.next_1016;
    }
}
```

Gambar 2.18 (Langkah 4 HapusDLL)

5. Kemudian, pointer node sebelumnya diubah menjadi null sehingga hubungan dengan node terakhir terputus dan node terakhir berhasil dihapus dari linked list.

```
// Update pointer previous node
if (curr_1016.prev_1016 != null) {
curr_1016.prev_1016.next_1016 = null;
}
return head_1016;
```

Gambar 2.19 (Langkah 5 HapusDLL)

6. Buat method delPos_1016() untuk menghapus node pada posisi tertentu. Program terlebih dahulu mengecek apakah linked list kosong sebelum proses penghapusan dilakukan.

```
// Fungsi menghapus node posisi tertentu
public static NodeDLL_2511531016 delPos_1016 (NodeDLL_2511531016
head_1016, int pos_1016) {
// Jika DLL kosong
if (head_1016 == null) {
return head_1016;
```

Gambar 2.20 (Langkah 6 HapusDLL)

7. Telusuri node hingga mencapai posisi yang ingin dihapus menggunakan perulangan for, lalu mengecek apakah posisi tersebut tersedia pada linked list.

```
NodeDLL_2511531016 curr_1016 = head_1016;
// Telusuri sampai ke node yang akan dihapus
for (int i_1016 = 1; curr_1016 != null && i_1016 < pos_1016;
++i_1016) {
curr_1016 = curr_1016.next_1016;
}
// Jika posisi tidak ditemukan
if (curr_1016 == null) {
return head_1016;
```

Gambar 2.21 (Langkah 7 HapusDLL)

8. Setelah node ditemukan, program memperbarui pointer next_1016 dan prev_1016 agar node yang dipilih terlepas dari rangkaian DLL.

```
// Update pointer
if (curr_1016.prev_1016 != null) {
    curr_1016.prev_1016.next_1016 = curr_1016.next_1016;
}
if (curr_1016.next_1016 != null) {
    curr_1016.next_1016.prev_1016 = curr_1016.prev_1016;
}
```

Gambar 2.22 (Langkah 8 HapusDLL)

9. Jika node yang dihapus merupakan head, maka head dipindahkan ke node berikutnya agar linked list tetap tersusun dengan benar.

```
// Jika yang dihapus head
if (head_1016 == curr_1016) {
    head_1016 = curr_1016.next_1016;
}
return head_1016;
```

Gambar 2.23 (Langkah 9 HapusDLL)

10. Selanjutnya, buat method printList_1016() untuk mencetak seluruh isi DLL dengan menelusuri node satu per satu dari head hingga akhir list.

```
// Fungsi mencetak DLL
public static void printList_1016 (NodeDLL_2511531016 head_1016) {
    NodeDLL_2511531016 curr_1016 = head_1016;
    while (curr_1016 != null) {
        System.out.print(curr_1016.data_1016 + " ");
        curr_1016 = curr_1016.next_1016;
    }
    System.out.println();
}
```

Gambar 2.24 (Langkah 10 HapusDLL)

11. Pada method main, buat DLL awal yang terdiri dari node 1, 2, 3, 4, dan 5 dengan hubungan pointer next_1016 dan prev_1016.

```

public static void main(String[] args) {
// Buat sebuah DLL
NodeDLL_2511531016 head_1016 = new NodeDLL_2511531016(1);
head_1016.next_1016 = new NodeDLL_2511531016(2);
head_1016.next_1016.next_1016 = new NodeDLL_2511531016(3);
head_1016.next_1016.next_1016.prev_1016 = head_1016.next_1016;
head_1016.next_1016.next_1016.next_1016 = new NodeDLL_2511531016(4);
head_1016.next_1016.next_1016.next_1016.prev_1016 =
head_1016.next_1016.next_1016;
head_1016.next_1016.next_1016.next_1016.next_1016 = new
NodeDLL_2511531016(5);
head_1016.next_1016.next_1016.next_1016.next_1016.prev_1016 =
head_1016.next_1016.next_1016.next_1016;

```

Gambar 2.25 (Langkah 11 HapusDLL)

12. Terakhir, tampilkan DLL awal, kemudian lakukan beberapa operasi penghapusan seperti menghapus head, menghapus node terakhir, dan menghapus node pada posisi kedua lalu cetak hasil akhirnya.

```

System.out.print("DLL Awal: ");
printList_1016(head_1016);

System.out.print("Setelah head dihapus: ");
head_1016 = delHead_1016(head_1016);
printList_1016(head_1016);

System.out.print("Setelah node terakhir dihapus: ");
head_1016 = dellast_1016(head_1016);
printList_1016(head_1016);

System.out.print("menghapus node ke 2: ");
head_1016 = delPos_1016(head_1016, 2);

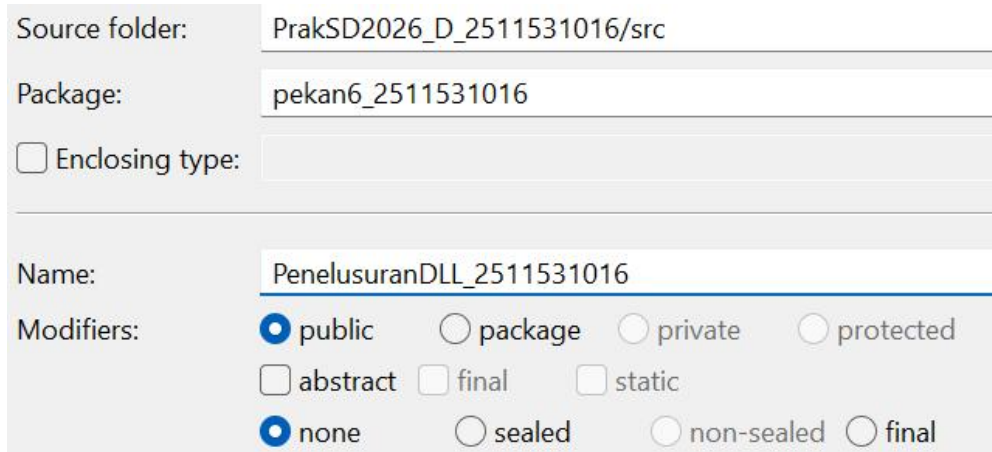
printList_1016(head_1016);

```

Gambar 2.26 (Langkah 12 HapusDLL)

2.3.4 PenelusuranDLL_2511531016

1. Membuat kelas baru bernama *PenelusuranDLL_2511531016* dalam *package pekan6_2511531016*.



Source folder:	PrakSD2026_D_2511531016/src
Package:	pekan6_2511531016
☐ Enclosing type:	
Name:	PenelusuranDLL_2511531016
Modifiers:	☒ public ☐ package ☐ private ☐ protected ☐ abstract ☐ final ☐ static ☒ none ☐ sealed ☐ non-sealed ☐ final

Gambar 2.27 (Langkah 1 PenelusuranDLL)

2. Program dimulai dengan deklarasi package dan class. Lalu, buat method `forwardTraversal_1016()` untuk melakukan penelusuran maju pada DLL dimulai dari head hingga node terakhir sambil menampilkan data setiap node.

```
package pekan6_2511531016;

public class PenelusuranDLL_2511531016 {
    // Fungsi penelusuran maju
    static void forwardTraversal_1016 (NodeDLL_2511531016 head_1016) {
        // Memulai penelusuran dari head
        NodeDLL_2511531016 curr_1016 = head_1016;
        // Lanjutkan sampai akhir
        while (curr_1016 != null) {
            // Print data
            System.out.print(curr_1016.data_1016 + " <-> ");
        }
    }
}
```

Gambar 2.28 (Langkah 2 PenelusuranDLL)

3. Selanjutnya, pointer `curr_1016` dipindahkan ke node berikutnya menggunakan `next_1016` hingga seluruh node selesai ditelusuri.

```
// Pindah ke node berikutnya
curr_1016 = curr_1016.next_1016;
}
// Print spasi
System.out.println();
```

Gambar 2.29 (Langkah 3 PenelusuranDLL)

4. Kemudian, buat method `backwardTraversal_1016()` untuk melakukan penelusuran mundur dimulai dari tail menuju head dengan bantuan pointer `prev_1016`.

```
// Fungsi penelusuran mundur
static void backwardTraversal_1016 (NodeDLL_2511531016 tail_1016) {
// Mulai dari akhir
NodeDLL_2511531016 curr_1016 = tail_1016;
// Lanjut sampai head
while (curr_1016 != null) {
// Cetak data
System.out.print(curr_1016.data_1016 + " <-> ");
```

Gambar 2.30 (Langkah 4 PenelusuranDLL)

5. Setelah itu, pointer `curr_1016` digeser ke node sebelumnya menggunakan `prev_1016` sampai mencapai head dari linked list.

```
// Pindah ke node sebelumnya
curr_1016 = curr_1016.prev_1016;
}
// Cetak spasi
System.out.println();
```

Gambar 2.31 (Langkah 5 PenelusuranDLL)

6. Pada method `main`, buat tiga buah node yaitu `head_1016`, `second_1016`, dan `third_1016` sebagai isi awal DLL.

```
public static void main(String[] args) {
// Cetak DLL
NodeDLL_2511531016 head_1016 = new NodeDLL_2511531016(1);
NodeDLL_2511531016 second_1016 = new NodeDLL_2511531016(2);
```

```
NodeDLL_2511531016 third_1016 = new NodeDLL_2511531016(3);
```

Gambar 2.32 (Langkah 6 PenelusuranDLL)

7. Lalu, setiap node dihubungkan menggunakan pointer next_1016 dan prev_1016 sehingga terbentuk hubungan dua arah antar node pada DLL.

```
head_1016.next_1016 = second_1016;  
second_1016.prev_1016 = head_1016;  
second_1016.next_1016 = third_1016;  
third_1016.prev_1016 = second_1016;
```

Gambar 2.33 (Langkah 7 PenelusuranDLL)

8. Terakhir, tampilkan hasil penelusuran maju menggunakan method forwardTraversal_1016() dan penelusuran mundur menggunakan method backwardTraversal_1016().

```
System.out.println("Penelusuran maju:");  
forwardTraversal_1016(head_1016);  
  
System.out.println("Penelusuran mundur:");  
backwardTraversal_1016(third_1016);
```

Gambar 2.34 (Langkah 8 PenelusuranDLL)

2.4 Analisis Hasil

Berdasarkan hasil praktikum yang telah dilakukan, implementasi Double Linked List pada program berhasil menunjukkan cara kerja struktur data dua arah dengan baik. Hal ini terlihat dari penggunaan pointer `next_1016` dan `prev_1016` yang memungkinkan setiap node saling terhubung ke node berikutnya maupun node sebelumnya. Pada proses penambahan data, program mampu menambahkan node di awal, di akhir, maupun pada posisi tertentu tanpa merusak susunan linked list yang sudah ada. Selain itu, proses penghapusan node juga berjalan sesuai tujuan, baik untuk menghapus head, node terakhir, maupun node pada posisi tertentu. Perubahan isi linked list setelah setiap operasi dilakukan menunjukkan bahwa pointer antar node telah diperbarui dengan benar sehingga struktur data tetap konsisten.

Selain operasi penambahan dan penghapusan, praktikum juga memperlihatkan proses penelusuran data pada Double Linked List secara maju dan mundur. Penelusuran maju dilakukan dari head menuju tail menggunakan pointer `next_1016`, sedangkan penelusuran mundur dilakukan dari tail menuju head menggunakan pointer `prev_1016`. Dari hasil output yang diperoleh, terlihat bahwa hubungan dua arah pada DLL memberikan fleksibilitas lebih dibanding Single Linked List karena data dapat diakses dari kedua sisi. Secara keseluruhan, praktikum ini membantu memahami bagaimana mekanisme kerja Double Linked List diterapkan dalam program Java, terutama dalam pengelolaan pointer antar node agar proses manipulasi data dapat dilakukan dengan lebih efisien dan terstruktur.

BAB III

KESIMPULAN

3.1 Hasil Praktikum

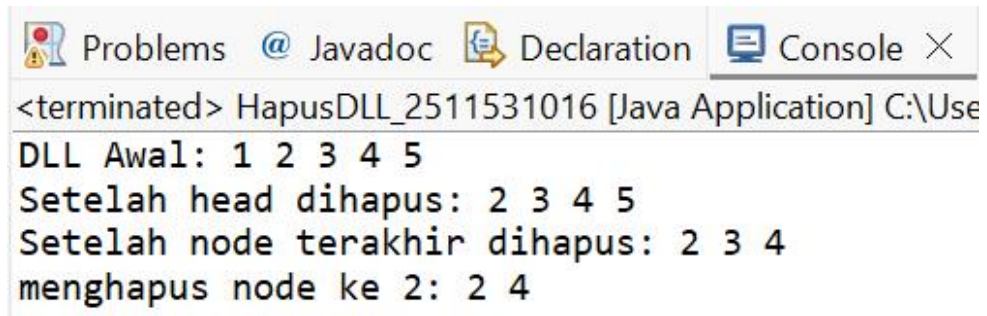
3.1.1 InsertDLL_2511531016

Gambar 3.1 (Hasil Praktikum InsertDLL)

Output program tersebut terlihat bahwa awalnya DLL berisi data 2 <-> 3 <-> 5, yang menunjukkan bahwa setiap node saling terhubung menggunakan pointer dua arah. Setelah dilakukan penambahan simpul 1 di awal, data berubah menjadi 1<->2 <-> 3 <-> 5, menandakan bahwa node baru berhasil dijadikan head baru tanpa mengganggu hubungan node lainnya.

Selanjutnya, program menambahkan simpul 6 di akhir linked list sehingga urutannya menjadi 1 <-> 2 <-> 3 <-> 5 <-> 6. Pada proses terakhir, node bernilai 4 disisipkan di posisi ke-4 dan hasil output berubah menjadi 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6. Dari perubahan tersebut dapat dilihat bahwa operasi penambahan node pada awal, akhir, maupun posisi tertentu berhasil dilakukan dengan benar karena susunan data tetap terhubung secara berurutan.

3.1.2 HapusDLL_2511531016



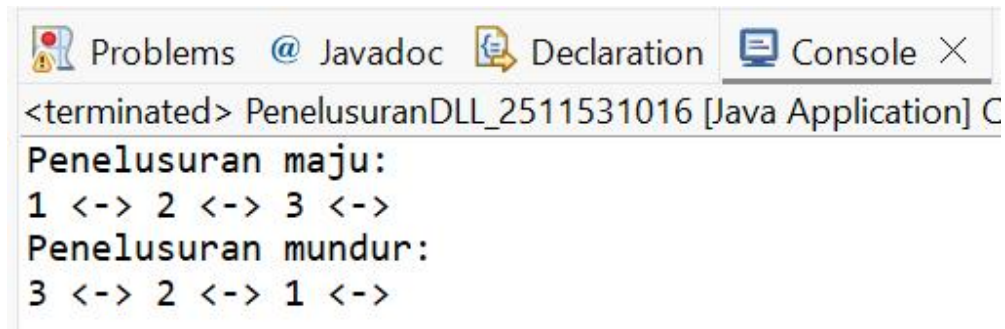
```
<terminated> HapusDLL_2511531016 [Java Application] C:\Use
DLL Awal: 1 2 3 4 5
Setelah head dihapus: 2 3 4 5
Setelah node terakhir dihapus: 2 3 4
menghapus node ke 2: 2 4
```

Gambar 3.2 (Hasil Praktikum HapusDLL)

Output di atas terlihat bahwa awalnya DLL berisi data 1 2 3 4 5 yang tersusun secara berurutan. Setelah method `delHead_1016()` dijalankan, node pertama berhasil dihapus sehingga head berpindah ke node berikutnya dan isi linked list berubah menjadi 2 3 4 5. Selanjutnya, program menghapus node terakhir menggunakan method `delLast_1016()`, sehingga data 5 tidak lagi tampil dan list menjadi 2 3 4.

Pada proses terakhir, program menghapus node pada posisi ke-2 menggunakan method `delPos_1016()`. Karena posisi kedua berisi data 3, maka node tersebut dihapus dan hasil akhirnya menjadi 2 4. Dari output tersebut terlihat bahwa setiap proses penghapusan berhasil memperbarui hubungan antar node dengan baik sehingga struktur DLL tetap tersusun dengan benar setelah terjadi perubahan data.

3.1.3 PenelusuranDLL_2511531016



```
<terminated> PenelusuranDLL_2511531016 [Java Application] C
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

Gambar 3.3 (Hasil Praktikum PenelusuranDLL)

Output program menunjukkan bahwa proses penelusuran maju menampilkan data 1 <-> 2 <-> 3, yang berarti traversal dimulai dari head menuju node terakhir menggunakan pointer next_1016. Urutan data yang tampil menunjukkan bahwa setiap node berhasil terhubung secara berurutan dari depan ke belakang pada DLL.

Setelah itu, program melakukan penelusuran mundur dan menghasilkan output 3 <-> 2 <-> 1, yang menandakan traversal dilakukan dari tail menuju head menggunakan pointer prev_1016. Hasil tersebut memperlihatkan bahwa setiap node pada DLL memiliki hubungan dua arah, sehingga data dapat diakses baik secara maju maupun mundur tanpa perlu mengulang penelusuran dari awal.

3.2 Saran Pengembangan

Sebagai pengembangan dari praktikum yang telah dilakukan, program DLL yang telah dibuat dapat ditambahkan fitur pencarian data dan pengurutan node agar pengguna lebih mudah mengelola isi linked list. Misalnya, program dapat dibuat untuk mencari data tertentu berdasarkan nilai node, kemudian menampilkan posisi data tersebut dalam list. Selain itu, dapat juga ditambahkan method sorting seperti ascending atau descending sehingga data dalam linked list dapat tersusun lebih rapi secara otomatis. Dengan adanya fitur tambahan tersebut, program tidak hanya berfungsi untuk manipulasi dasar node, tetapi juga lebih mendekati penerapan struktur data pada sistem nyata.

Selain penambahan fitur, tampilan program juga dapat dikembangkan menjadi lebih interaktif dengan menggunakan menu input berbasis Scanner, sehingga pengguna bisa memilih operasi tambah, hapus, atau penelusuran secara langsung melalui terminal. Penanganan error juga bisa diperbaiki, misalnya dengan memberikan validasi saat posisi node yang dimasukkan melebihi panjang linked list atau ketika linked list dalam kondisi kosong. Pengembangan ini akan membuat program menjadi lebih aman digunakan dan membantu memperkuat pemahaman mengenai cara kerja pointer pada DLL dalam berbagai kondisi.

DAFTAR PUSTAKA

- [1] A. Saputra dan R. Kurniawan, “Implementasi Double Linked List pada Pengelolaan Data Menggunakan Bahasa Pemrograman Java,” *Jurnal Informatika dan Teknologi Komputer*, 2023.
- [2] D. Pratama dan Y. Febriani, “Penerapan Struktur Data Double Linked List dalam Sistem Pengolahan Data Dinamis,” *Jurnal Sistem Informasi dan Komputer*, 2024.
- [3] M. Rizki, F. Hidayat, dan N. Aulia, “Analisis Implementasi Double Linked List Menggunakan Java pada Pembelajaran Struktur Data,” *Jurnal Teknologi Informasi dan Pendidikan*, 2023.

TUGAS PRAKTIKUM PEKAN 6

Playlist Lagu Sederhana

1. Deskripsi Tugas

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

2. Kode Program

2.1 Lagu_2511531016

```
package pekan6_2511531016;

public class Lagu_2511531016 {
    String judul_1016;
    String penyanyi_1016;
    Lagu_2511531016 next_1016;
    Lagu_2511531016 prev_1016;

    // konstruktor
    public Lagu_2511531016(String judul_1016, String penyanyi_1016) {
        this.judul_1016 = judul_1016;
        this.penyanyi_1016 = penyanyi_1016;
        this.next_1016 = null;
        this.prev_1016 = null;
    }

    // getter
    public String getJudul_1016() {
        return judul_1016;
    }
    public String getPenyanyi_1016() {
        return penyanyi_1016;
    }

    // setter
    public void setJudul_1016(String judul_1016) {
        this.judul_1016 = judul_1016;
    }
    public void setPenyanyi_1016(String penyanyi_1016) {
        this.penyanyi_1016 = penyanyi_1016;
    }
}
```

Penjelasan :

Kelas Lagu_2511531016 dipakai sebagai node pada Double Linked List. Di dalamnya terdapat data lagu seperti judul dan penyanyi, lalu ada pointer next_1016 dan prev_1016 yang berfungsi untuk menghubungkan satu lagu dengan lagu lainnya. Pointer next_1016 mengarah ke lagu berikutnya, sedangkan prev_1016 mengarah ke lagu sebelumnya. Karena menggunakan dua pointer, playlist bisa ditelusuri dari depan ke belakang maupun dari belakang ke depan. Saat objek lagu dibuat, konstruktor langsung mengisi data judul dan penyanyi, sementara pointer awalnya bernilai null.

Selain itu, kelas ini juga menyediakan getter dan setter untuk setiap atribut. Getter digunakan untuk mengambil data lagu, sedangkan setter dipakai kalau data ingin diubah. Walaupun program sebenarnya masih bisa berjalan tanpa getter dan setter, penggunaan method ini membuat struktur program jadi lebih rapi dan sesuai konsep ADT. Dari praktikum ini juga terlihat bahwa kelas node punya peran penting karena menjadi dasar penghubung antar data pada Double Linked List.

2.2 Musik_2511531016

```
package pekan6_2511531016;
import java.util.Scanner;

public class Musik_2511531016 {
    Lagu_2511531016 head_1016 = null;
    Lagu_2511531016 tail_1016 = null;

    // tambah lagu di akhir playlist
    public void tambahLagu_1016(String judul_1016, String penyanyi_1016) {
        Lagu_2511531016 laguBaru_1016 = new Lagu_2511531016(judul_1016, penyanyi_1016);
        if (head_1016 == null) {
            head_1016 = laguBaru_1016;
            tail_1016 = laguBaru_1016;
        } else {
            tail_1016.next_1016 = laguBaru_1016;
            laguBaru_1016.prev_1016 = tail_1016;
            tail_1016 = laguBaru_1016;
        }
        System.out.println("Lagu berhasil ditambahkan!");
    }

    // hapus lagu pertama
    public void hapusLaguAwal_1016() {
        if (head_1016 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        System.out.println("Lagu yang dihapus: " + head_1016.judul_1016);

        if (head_1016 == tail_1016) {
            head_1016 = null;
            tail_1016 = null;
        } else {
            head_1016 = head_1016.next_1016;
            head_1016.prev_1016 = null;
        }
    }

    // tampil maju
    public void tampilMaju_1016() {
        if (head_1016 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511531016 curr_1016 = head_1016;
        System.out.println("Playlist Maju:");
        while (curr_1016 != null) {
            System.out.println("Judul : " + curr_1016.judul_1016);
            System.out.println("Penyanyi : " + curr_1016.penyanyi_1016);
            curr_1016 = curr_1016.next_1016;
        }
    }

    // tampil mundur
    public void tampilMundur_1016() {
        if (tail_1016 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511531016 curr_1016 = tail_1016;
        System.out.println("Playlist Mundur:");
        while (curr_1016 != null) {
            System.out.println("Judul : " + curr_1016.judul_1016);
            System.out.println("Penyanyi : " + curr_1016.penyanyi_1016);
            curr_1016 = curr_1016.prev_1016;
        }
    }
}
```

```

// cari lagu
public void carilagu_1016(String judul_1016) {
    Lagu_2511531016 curr_1016 = head_1016;
    while (curr_1016 != null) {
        if (curr_1016.judul_1016.equalsIgnoreCase(judul_1016)) {
            System.out.println("Lagu ditemukan!");
            System.out.println("Judul : " + curr_1016.judul_1016);
            System.out.println("Penyanyi : " + curr_1016.penyanyi_1016);
            return;
        }
        curr_1016 = curr_1016.next_1016;
    }
    System.out.println("Lagu tidak ditemukan!");
}

public static void main(String[] args) {
    Scanner input_1016 = new Scanner(System.in);
    Musik_2511531016 playlist_1016 = new Musik_2511531016();
    int choice_1016;

    do {
        System.out.println("\n=== Playlist Musik NIM: 2511531016 ===");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");

        choice_1016 = input_1016.nextInt();
        input_1016.nextLine();

        switch (choice_1016) {
            case 1:
                System.out.print("Judul Lagu: ");
                String judul_1016 = input_1016.nextLine();
                System.out.print("Penyanyi: ");
                String penyanyi_1016 = input_1016.nextLine();
                playlist_1016.tambahLagu_1016(judul_1016, penyanyi_1016);
                break;
            case 2:
                playlist_1016.hapusLaguAwal_1016();
                break;
            case 3:
                playlist_1016.tampilMaju_1016();
                break;
            case 4:
                playlist_1016.tampilMundur_1016();
                break;
            case 5:
                System.out.print("Masukkan judul lagu yang dicari: ");
                String cari_1016 = input_1016.nextLine();
                playlist_1016.carilagu_1016(cari_1016);
                break;
            case 6:
                System.out.println("Program selesai.");
                break;
            default:
                System.out.println("Pilihan tidak valid!");
        }
    } while (choice_1016 != 6);
    input_1016.close();
}
}

```

Penjelasan :

Kelas Musik_2511531016 berfungsi sebagai pengatur utama playlist lagu. Pada kelas ini terdapat pointer head_1016 yang menunjuk lagu pertama dan tail_1016 yang menunjuk lagu terakhir. Method tambahLagu_1016() digunakan untuk menambahkan lagu baru ke bagian akhir playlist. Kalau playlist masih kosong, lagu pertama langsung dijadikan head dan tail. Tetapi kalau playlist sudah memiliki isi, maka pointer antar node akan dihubungkan supaya urutan lagu tetap tersusun dengan benar. Dari proses ini terlihat bagaimana Double Linked List bekerja menggunakan hubungan dua arah antar node.

Program juga memiliki beberapa operasi lain seperti menghapus lagu pertama, menampilkan playlist maju, menampilkan playlist mundur, dan mencari lagu berdasarkan judul. Penelusuran maju dilakukan dari head ke tail memakai pointer next_1016, sedangkan penelusuran mundur dilakukan dari tail ke head memakai pointer prev_1016. Pada bagian main, program menggunakan menu sederhana dengan switch-case supaya pengguna bisa memilih fitur yang ingin dijalankan. Secara keseluruhan, program ini membantu memahami cara kerja Double Linked List tidak hanya dari teori, tetapi juga dari bagaimana pointer dihubungkan dan dipakai dalam pengelolaan data playlist lagu.

3. Output Kode Program

```
Problems @ Javadoc Declaration Console X
<terminated> Musik_2511531016 [Java Application] C:\Users\U

=== Playlist Musik NIM: 2511531016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: disarankan di bandung
Penyanyi: Dongker, Jason Ranti
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: 33x
Penyanyi: Perunggu
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
Playlist Maju:
Judul : disarankan di bandung
Penyanyi : Dongker, Jason Ranti
Judul : 33x
Penyanyi : Perunggu

=== Playlist Musik NIM: 2511531016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
Playlist Mundur:
Judul : 33x
Penyanyi : Perunggu
Judul : disarankan di bandung
Penyanyi : Dongker, Jason Ranti
```

=== Playlist Musik NIM: 2511531016 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 5

Masukkan judul lagu yang dicari: 33x

Lagu ditemukan!

Judul : 33x

Penyanyi : Perunggu

=== Playlist Musik NIM: 2511531016 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 2

Lagu yang dihapus: disarankan di bandung

=== Playlist Musik NIM: 2511531016 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 5

Masukkan judul lagu yang dicari: disarankan di bandung

Lagu tidak ditemukan!

=== Playlist Musik NIM: 2511531016 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 6

Program selesai.

Penjelasan :

Output program di atas menunjukkan cara kerja playlist musik yang dibuat menggunakan struktur data double linked list. Awalnya, pengguna menambahkan dua lagu ke dalam playlist, yaitu “disarankan di bandung” dari Dongker dan Jason Ranti, lalu lagu “33x” dari Perunggu. Setelah lagu ditambahkan, program bisa menampilkan daftar playlist secara maju dan mundur. Saat ditampilkan maju, lagu muncul sesuai urutan penambahan, sedangkan saat ditampilkan mundur urutannya jadi kebalik dari lagu terakhir ke lagu pertama. Dari hasil ini terlihat bahwa setiap node saling terhubung ke node berikutnya dan sebelumnya, sehingga data bisa diakses dari dua arah.

Program juga dapat melakukan pencarian lagu berdasarkan judul. Ketika pengguna mencari lagu “33x”, data lagu berhasil ditemukan dan ditampilkan. Setelah itu, pengguna menghapus lagu pertama dari playlist sehingga lagu “disarankan di bandung” terhapus dari linked list. Saat lagu tersebut dicari kembali, program menampilkan pesan bahwa lagu tidak ditemukan karena node-nya sudah dihapus dari playlist. Dari keseluruhan output, program sudah berhasil menjalankan operasi dasar double linked list seperti tambah data, hapus data, traversal maju-mundur, dan pencarian data dengan baik.